KUBECON NA 2026 · KUBEVELA

One interface to rule them all

One application.yaml. Any IaC. Any cloud.

aws-cue aws-devkit aws-ack gcp-devkit

Building from the developer’s side

01 · WRITE

My application.yaml

One file declares my app, its traits and its bucket.

02 · DEPLOY

My deployment

The same properties go to every stage. Nothing to edit.

03 · RUN

My app code

The platform provides the abstraction, so the code stays as it is.

One interface, from the developer’s side.

My application.yaml

```yaml
components:
  - name: product-images
    type: bucket-aws-cue
    properties:
      name: product-images
      region: us-west-2
      projectName: kubecon-na-2026
```

PROPERTIES

Mine. Never edited.

They go to every stage as written.

TYPE

The platform's choice.

An override policy swaps it per stage.

My deployment is not changing

Four stages, one thing changes each time. Red marks it.

`aws-cue`	AWS S3 via Crossplane, CUE	Same properties
`aws-devkit`	AWS S3 via Crossplane, DevKit	Same properties
`aws-ack`	AWS S3 via ACK, DevKit	Same properties
`gcp-devkit`	GCP GCS via Crossplane, DevKit	Same properties

PLATFORM ENGINEER

One component, two ways to write it

bucket-aws-cue

CUE

The language KubeVela definitions have always used.

bucket-aws-ack · Go with DevKit

```
defkit.NewComponent("bucket-aws-ack").
    Workload("s3.services.k8s.aws/v1alpha1", "Bucket").
    Params(p.all()...).
    Template(p.renderACK).
    HealthPolicy(bucketHealth(ackSynced))
```

Same resources out. The developer's file can't tell them apart.

PLATFORM ENGINEER

Why Go for components

Familiar

Many engineers already know Go better than CUE.

A full ecosystem

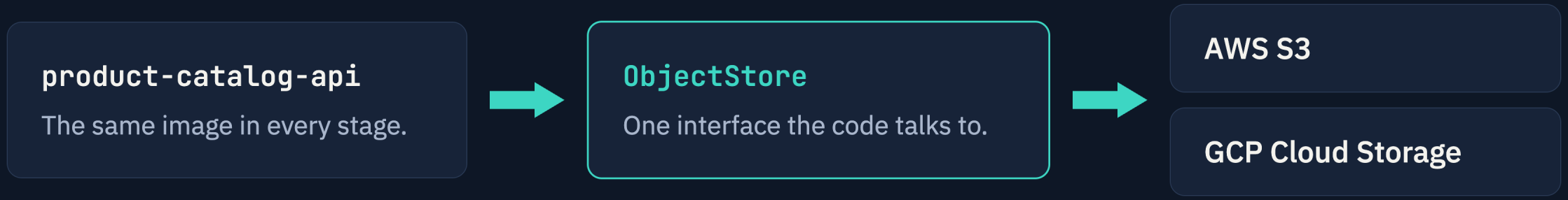
Modules, IDE support, linters and the libraries they know.

Real tests

go test, with no cluster. It checks Go renders the same as CUE.

DevKit is new in KubeVela.

Even the app stays the same



The platform sets STORAGE_PROVIDER. The app never calls a cloud SDK outside its backend.

Independent from the IaC

`aws-devkit`

Crossplane

A managed resource from provider-aws-s3 creates the bucket.



`aws-ack`

ACK

The ACK S3 controller creates it from a Bucket in the stage's namespace.

Same properties in. Same bucket out.

Independent from the cloud

aws-ack

AWS S3

A bucket in Floci, our local S3.



gcp-devkit

GCP GCS

A bucket in floci-gcp, our local Cloud Storage.

The bucket claim stays. The platform handles the cloud.

One interface.

One application.yaml for the developer.

Any IaC: Crossplane or ACK.

Any cloud: AWS or GCP.

At the booth: approve a stage, watch its bucket appear.